

MQTT, HTTP, SPI, and I2C — Explained from First Principles

0. The Big Picture

Every one of these four things solves the same root problem — "**how does one piece of hardware or software get data to another?**" — but they operate at completely different altitudes:

MQTT	→	Internet-scale messaging (Layer 7)
HTTP	→	Internet-scale request/response (Layer 7)

"Two computers on opposite sides of the planet talk"

I2C	→	Chip-to-chip on a circuit board
SPI	→	Chip-to-chip on a circuit board

"Two chips 2cm apart on the same PCB talk"

HTTP and MQTT run **on top of** networks like Wi-Fi/Ethernet (which themselves run on top of TCP/IP). SPI and I2C are much more primitive — they are the actual **electrical wires and voltage pulses** between chips, with no "internet" involved at all. A typical IoT device uses *both* worlds: a temperature sensor talks to a microcontroller via I2C, and that microcontroller talks to the cloud via MQTT or HTTP.

Keep that mental map in your head — it'll make everything below click into place.

1. HTTP — HyperText Transfer Protocol

First principles

HTTP exists to answer one question: "**Client, you asked for something — here it is.**" It's a **request/response** protocol. Nothing happens until someone asks.

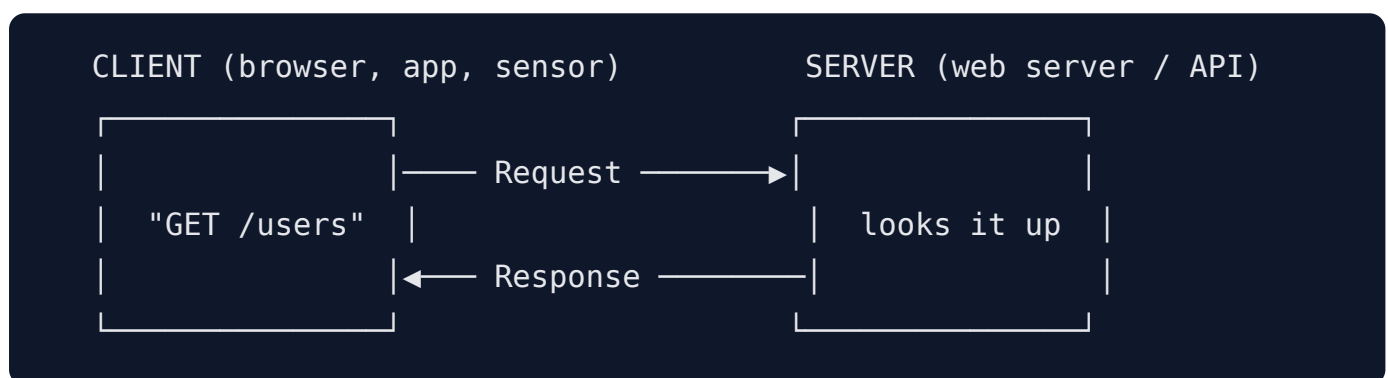
The analogy: ordering at a counter

Imagine a fast-food counter.

- You (the **client**) walk up and say: *"One burger, no onions."*
- The cashier (the **server**) prepares it and hands it back with a receipt.
- Then you walk away. The cashier doesn't know you, doesn't remember your order, and won't call you later. Every time you want something, you walk back up and ask again from scratch.

That "forgetting" is **statelessness** — the defining trait of HTTP. Each request stands completely alone.

Architecture



Anatomy of a request

```
GET /weather?city=Delhi HTTP/1.1      ← Method + Path + Version
Host: api.example.com                 ← Headers (metadata)
Authorization: Bearer xyz123
Accept: application/json

(optional body – used in POST/PUT)
```

Method = verb (what you want to do):

- **GET** — fetch something ("show me the menu")
- **POST** — create something ("place a new order")
- **PUT / PATCH** — update something ("change my order")
- **DELETE** — remove something ("cancel my order")

Anatomy of a response

```
HTTP/1.1 200 OK
```

← Status line

```
Content-Type: application/json
```

```
{"temp": 31, "city": "Delhi"}
```

← Body (the actual data)

Status codes are the receipt's fine print:

- **2xx** = success (200 OK, 201 Created)
- **3xx** = redirect ("go ask over there instead")
- **4xx** = *you* messed up (404 Not Found, 401 Unauthorized)
- **5xx** = *server* messed up (500 Internal Server Error)

Why it matters

HTTP is the backbone of the web and of most REST APIs. It's simple, human-readable, and universally supported — but because it's stateless and client-initiated, the **server can never push data to you on its own**. If you want live updates (a stock ticker, a chat message), plain HTTP has to fake it by repeatedly asking "anything new? anything new? anything new?" — called *polling*. That inefficiency is exactly the gap MQTT was built to fill.

2. MQTT — Message Queuing Telemetry Transport

First principles

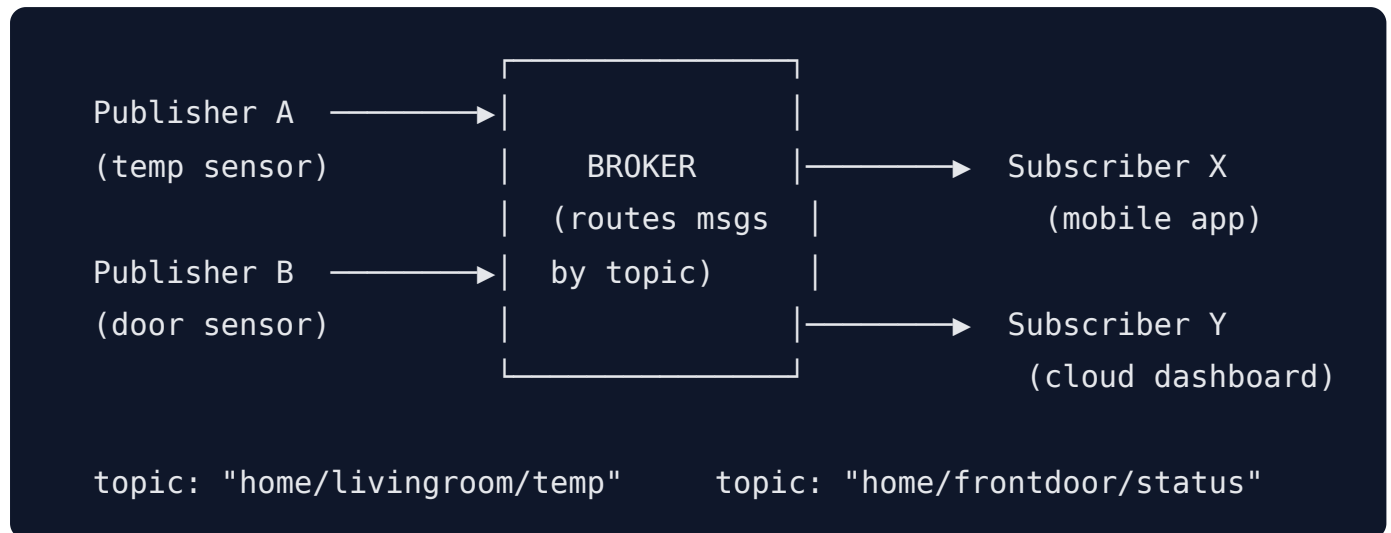
MQTT flips HTTP's model on its head. Instead of "ask and receive," it's "**announce and let interested parties listen**." This is called **publish/subscribe (pub/sub)**, and its whole purpose is to *decouple* the sender from the receiver — they never talk directly and don't even need to know the other exists.

The analogy: a radio station

- A radio station (the **publisher**) broadcasts on 98.5 FM. It has no idea who's listening, how many people, or whether anyone's tuned in at all.
- Listeners (**subscribers**) tune their radio to 98.5 FM (the **topic**) whenever they want.
- The tower relaying the broadcast to everyone's radios is the **broker**.

The station never sends anything directly to your radio — it just broadcasts into the air on a channel, and your radio decides which channel to listen to.

Architecture



Every message carries a **topic** — a slash-separated string like `home/livingroom/temperature`. Subscribers say "give me everything under `home/livingroom/#`" (the `#` is a wildcard meaning "everything below this"), and the broker handles all the routing.

Key concepts

- **Broker:** the central hub (e.g., Mosquitto, HiveMQ, AWS IoT Core) all clients connect to. No client ever talks directly to another client.
- **Topic:** a hierarchical label (`factory/floor1/machine3/temperature`) used to filter messages — not a fixed "channel" that has to be pre-created.
- **QoS (Quality of Service)** — how hard MQTT tries to guarantee delivery:
 - `QoS 0` — "fire and forget," at most once (fast, might lose messages)
 - `QoS 1` — "at least once" (might get duplicates, but never lost)
 - `QoS 2` — "exactly once" (slowest, but guaranteed no duplicates/loss)
- **Retained messages:** the broker can hold the *last* message on a topic and immediately hand it to anyone who subscribes later — like a radio station replaying the last song for someone who just tuned in.
- **Last Will and Testament (LWT):** a message the broker automatically publishes on your behalf if your connection drops unexpectedly — useful for "sensor X went offline" alerts.

Why it exists

MQTT was designed (originally for oil pipeline monitoring over unreliable satellite links) to be extremely lightweight — tiny header overhead, works on flaky networks, and lets a

battery-powered sensor push updates the instant something changes, instead of a server having to constantly poll it. This makes it the dominant protocol for **IoT**: thousands of sensors publishing tiny bits of data, and dashboards/cloud services subscribing to whatever they care about.

HTTP vs MQTT at a glance

	HTTP	MQTT
Model	Request/Response	Publish/Subscribe
Who initiates	Client always asks	Either side can push, anytime
Connection	Often short-lived (per request)	Long-lived, persistent
Overhead	Heavier (verbose headers)	Very lightweight
Best for	Web pages, REST APIs	Real-time IoT telemetry, many-to-many

Now let's drop down from "internet-scale" to "two chips on the same circuit board."

3. SPI — Serial Peripheral Interface

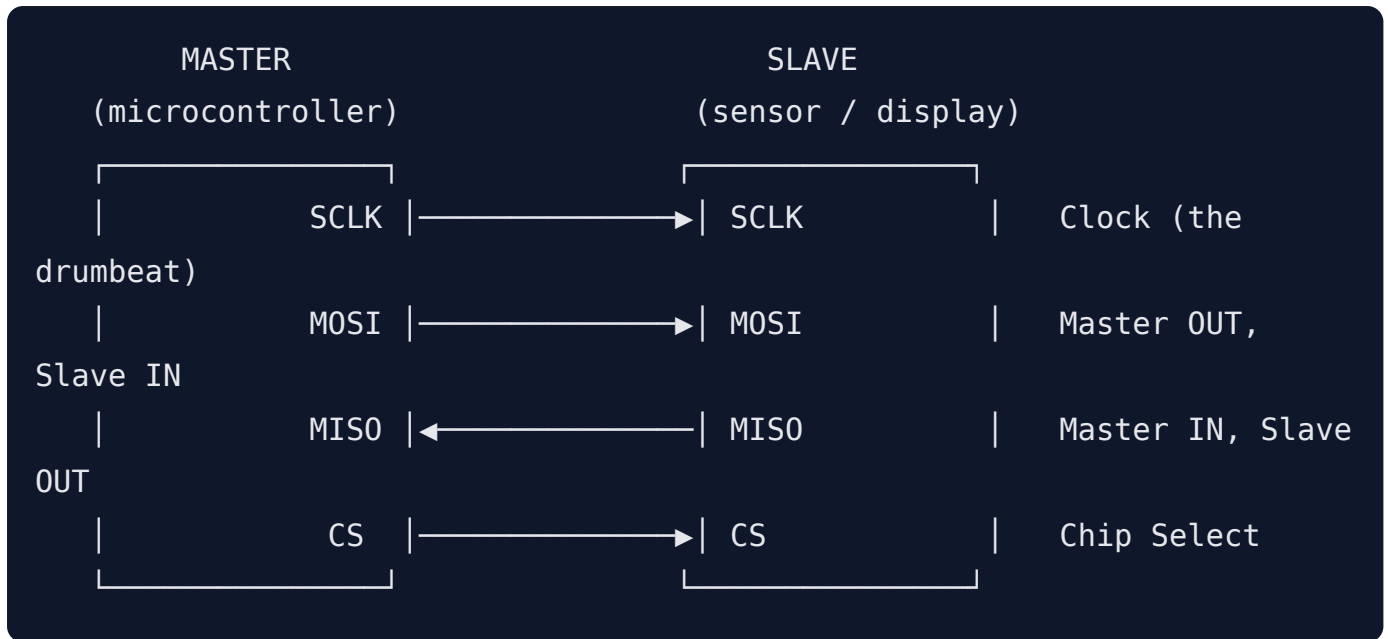
First principles

SPI is not a network protocol — it's a **physical wiring standard** for one chip to talk to another over very short distances (centimeters), typically a microcontroller talking to a sensor, display, or memory chip. There's no "packet," no "address book" in the internet sense — just raw wires and precisely timed voltage pulses.

The analogy: a drumbeat and a conveyor belt

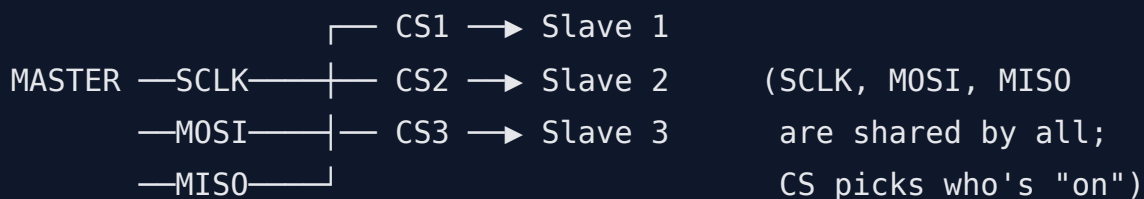
Imagine a factory foreman (the **master**, usually your microcontroller) who beats a drum at a steady rhythm. Every time the drum beats, exactly one item moves down the conveyor belt from the foreman to the worker, **and** one item moves from the worker back to the foreman — simultaneously, in both directions. The worker (the **slave**, e.g. a sensor chip) never speaks unless the foreman's drum is beating; it has no rhythm of its own.

The four wires



- **SCLK (Serial Clock)** — the master generates this; it's the metronome. One tick = one bit transferred.
- **MOSI (Master Out, Slave In)** — data flowing from master to slave.
- **MISO (Master In, Slave Out)** — data flowing from slave to master, at the *exact same time* as MOSI. This simultaneous two-way flow is called **full-duplex**.
- **CS / SS (Chip Select / Slave Select)** — the master pulls this wire low to say "I'm talking to *you* specifically." If you have five slave devices, you need five separate CS wires (one per device), all sharing the same SCLK/MOSI/MISO lines.

One master, multiple slaves:



Why it matters

SPI is **fast** (can run into tens of MHz) and simple to implement in hardware, which is why it's used for things needing high throughput at short range: SD cards, TFT/OLED displays, flash memory chips, high-speed ADCs. The tradeoff is it needs more wires (one extra CS per device) and there's no built-in addressing or error-checking — it trusts the wiring completely.

4. I2C — Inter-Integrated Circuit

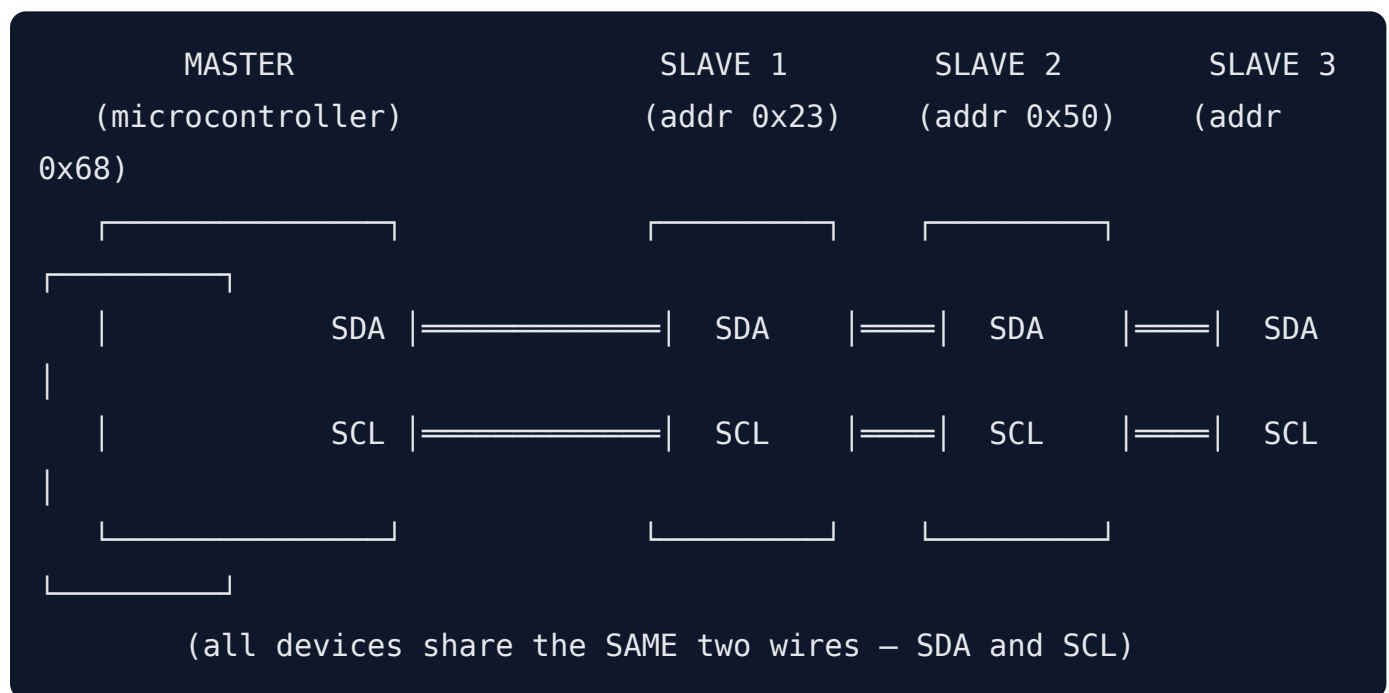
First principles

I2C solves the "too many wires" problem of SPI. Instead of a dedicated CS wire per device, every chip shares just **two wires total**, and each chip is identified by a numeric **address** — like a name, not a dedicated cable.

The analogy: a walkie-talkie party line

Imagine one shared radio channel that an entire room of people can hear. If you want to talk to "Priya" specifically, you say her name first — *"Priya, are you there?"* — and everyone else on the channel stays quiet and ignores the conversation, except Priya, who responds. Everyone shares the same channel; names (addresses) sort out who's being spoken to.

The two wires



- **SCL (Serial Clock)** — same idea as SPI's clock, the timing pulse (though I2C is slower — typically 100 kHz–400 kHz vs SPI's tens of MHz).
- **SDA (Serial Data)** — a *single* wire carrying data **both directions**, just not at the same instant. This is **half-duplex**: talk, then listen, then talk again — like a walkie-talkie where only one person can transmit at a time.

How addressing replaces extra wires

Every message on I2C starts with a **7-bit (or 10-bit) address** that names which chip should respond:

```
[START] [Address: 0x50] [R/W bit] [ACK] [Data byte] [ACK] ... [STOP]
```

- **START condition:** a special electrical signal meaning "attention, new conversation beginning."
- **Address + R/W bit:** "I want to talk to device 0x50, and I want to Read (or Write)."
- **ACK (Acknowledge):** the addressed slave pulls the line low to say "yes, that's me, I'm listening" — every other chip on the bus stays silent.
- **Data bytes:** the actual payload, one byte at a time, each followed by an ACK.
- **STOP condition:** "conversation over, bus is free."

This is why you can plug a dozen different sensors into the same two wires — as long as each has a unique address — and the master simply "calls out a name" before each exchange.

Why it matters

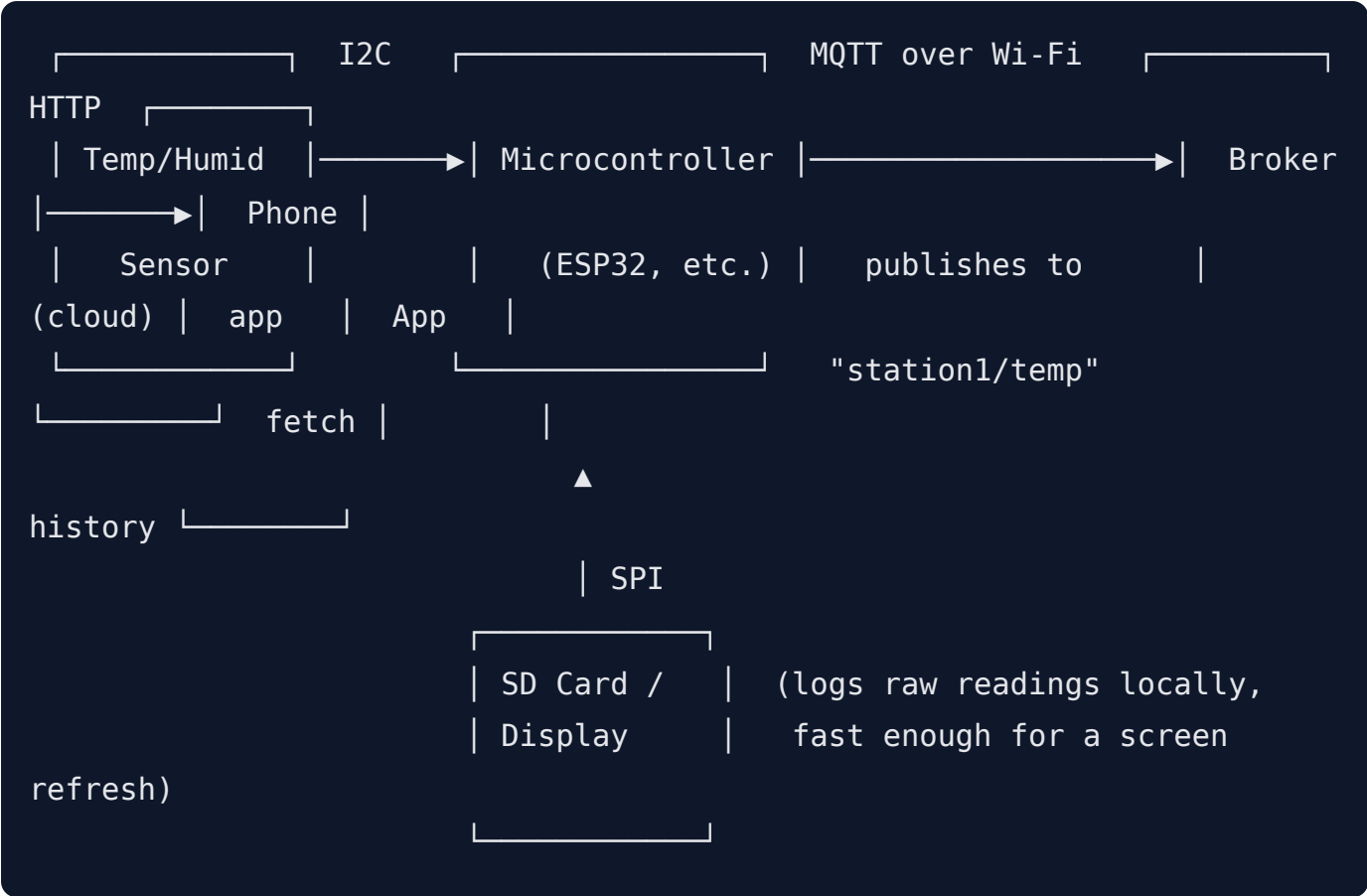
I2C trades speed for **simplicity of wiring**: only 2 wires total no matter how many devices you connect (SPI needs 3 + N). This makes it the go-to choice for connecting several low-speed sensors (temperature, humidity, accelerometers, RTC clocks) to one microcontroller without wiring chaos. The tradeoff is it's slower and, since it's half-duplex with shared wires, more chips on the bus means more chance of address collisions or bus contention.

SPI vs I2C at a glance

	SPI	I2C
Wires needed	4 + 1 per extra device	2 total, regardless of device count
Speed	Very fast (MHz–tens of MHz)	Slower (100–400 kHz typical)
Duplex	Full (send & receive simultaneously)	Half (one direction at a time)
Addressing	None — dedicated CS wire per device	Built-in numeric address per device
Best for	High-speed, few devices (displays, SD cards)	Many low-speed sensors on one bus

5. Putting it all together — a real device

Picture a smart weather station that reports data to a phone app:



- **I2C** connects the cheap, slow sensor chip to the microcontroller — few wires, good enough speed.
- **SPI** connects the same microcontroller to something that needs more bandwidth, like an SD card or a screen.
- **MQTT** lets the microcontroller *push* live readings to the cloud the instant they change, without the cloud having to constantly ask.
- **HTTP** is what the phone app uses to *pull* historical data or dashboards from the cloud on demand.

Four different protocols, four different jobs — each chosen because it's the right tool for that specific hop in the chain.

Quick-reference summary

Protocol	Layer	Model	Typical use
----------	-------	-------	-------------

HTTP	Internet	Request/Response	Web pages, REST APIs, on-demand data
MQTT	Internet	Publish/Subscribe	Real-time IoT telemetry, push notifications
SPI	Hardware bus	Master/Slave, full-duplex	Fast, short-range chip-to-chip (displays, SD cards)
I2C	Hardware bus	Master/Slave, half-duplex, addressed	Multiple low-speed sensors on 2 shared wires